GuidedTrack Toolbox: `gt`

Description

`gt` - a set of utilities to make it easier to work with GuidedTrack using files stored locally on your computer

Installation

Copy the `gt` script to a location on your path (commonly `/usr/local/bin/`).

Synopsis

```
gt [OPTION...] create
gt create
gt build
```

Commands

There are three things you could want to do with `gt`: creating programs, pushing updates to programs, and building programs.

Pushing updates: `gt push`

Pushes a set of local files to a GuidedTrack server as programs. File names must match the program names. If a file doesn't match a program on the server, that file will be skipped.

Environment

Which server the files get pushed to depends on the value of the `GT_ENV` environment variable. Possible values are:

- `development` - the files will be pushed to a local server, expected to run on port 3000 (http://localhost:3000)
- `stage` - the files will be pushed to the staging server at https://guidedtrack-stage.herokuapp.com
- `production` - the files will be pushed to the official GuidedTrack server at https://www.guidedtrack.com

Pushing to production is generally considered risky, as it will have an impact on real users, so you'll be warned when trying to do so and asked to confirm that you're sure you want to proceed.

Options

There are a couple of options you could pass into `gt push` to get slightly different behavior:

- `-o` - makes it so `gt push` pushes only files matching the selector provided

- `-b` - tells `gt push` to build all projects after pushing. The projects are listed in `.gt_projects`, one line per program. The program name must match an existing program on that server.

Examples

Push all files in the current directory to the development server

```
gt push
```

Push all files in the current directory to the staging server

```
GT_ENV=staging gt push
```

Push all files in the current directory to the production server and build

```
GT_ENV=production gt push -b
```

Push only files that end in "v2" to the production server and build

```
GT_ENV=production gt push -o *v2 -b
```

Creating programs: `gt create`

Creates a set of local files as programs to a GuidedTrack server. Each file in the current directory will be considered a program, so a program of the same name as the file will end up being created on the GuidedTrack server.

Examples

Rebuilding programs: `gt build`

Rebuilds all projects listed in the `.gt_projects` file. It assumes each line in the `.gt_projects` file corresponds to a program that is already created on the GuidedTrack server.

Example

Say you're working on an app called Appr and you've organized the whole project into four programs:

1. `Appr-intro`: an onboarding program that users run before signing up as a way to show them the value of the app
2. `Appr-dashboard`: the main program
3. `Appr-exercise`: a program linked to from the dashboard running the user through an intervention
4. `Appr-feedback`: a program linked to from the dashboard that allows your users to send you comments on their experience

It would be sensible to download the code of the above programs from GuidedTrack and place all the files into the same directory called `appr`. You'll end up with a folder on your computer with the following structure:

```
appr/
├── Appr-dashboard
├── Appr-exercise
├── Appr-feedback
└── Appr-intro
```

Your users will never run `Appr-exercise` directly - they'll always go through the dashboard to get to it. The only programs your users will need run directly are `Appr-dashboard` and `Appr-intro` - these are your projects that need to be rebuilt when there are changes. To make sure `gt build` and `gt push -b` do that, create a file `appr/.gt_projects` with the following contents:

```
Appr-dashboard
Appr-intro
```

You'll then be able to build those projects from the command line by using:

```
gt build
```

Or, in production:

```
GT_ENV=production gt build
```